# COLLEGE MULTIMODAL RAG

# Backend Implementation Plan — Updated

Includes agentic routing, task planning, and negative-result caching

**Purpose:** Backend-only, phase-by-phase implementation plan for the college assistant described in the uploaded implementation brief, expanded to include the missing lightweight agentic router behavior shown in the user's diagram.

**Build principle:** Start with a working text RAG pipeline, then add retrieval fallback, OCR, specialist agents, planning/routing, fault tolerance, and tests. Keep orchestration as plain Python functions initially; add frameworks only if needed.

## 1. Backend scope and architecture

Admin flow: upload → validate → extract PDF/TXT/image text → OCR scanned pages as needed → normalize and chunk → attach source metadata → embed → store in ChromaDB and a local text cache → mark indexed.

Student flow: message + optional screenshot → validate → lightweight agentic router → direct general response OR college RAG OR multi-task plan OR negative-cache check → specialist agent(s) → evidence validation → one combined answer with citations, warnings, and recovery status.

The router is a small orchestrator, not a separate autonomous agent framework. It decides the path and task execution plan; it does not decide whether college facts are true. Official indexed evidence remains the source of truth.

## 2. The missing router implementation — now included

The earlier backend plan included scope validation and task decomposition, but it did not spell out all four branches shown in the screenshot as an explicit, testable routing module. Implement this phase as a first-class backend feature.

| Route | Implementation behavior |
|---|---|
| General conversation | Recognize greetings, thanks, small talk, and suitable general questions. Respond directly using a simple handler or Ollama. Do not call embeddings, ChromaDB, BM25, or document retrieval. |
| College-specific query | Fees, registration dates, semester calendar, syllabus, course structure, campus rules, etc. must use RAG. Check the negative-result cache before doing retrieval. |
| Multiple tasks | Create bounded subtasks. Run independent tasks in parallel; run dependent tasks sequentially. Collect results and generate exactly one final response. |
| Known unanswered query | Check negative-result cache using normalized query/intent/filters and current document generation. Reuse a valid cached no-evidence result; otherwise retrieve normally. |
| Unclear/out of scope | Ask a concise clarification if ambiguous. Politely decline unrelated requests. Do not reject informal/slang wording just because it is phrased casually. |

## 3. Router contract and execution rules

Implement a deterministic, bounded router. Use simple rules for obvious greetings; use Ollama with a constrained Pydantic schema for ambiguous intent and task decomposition. Never let model output call arbitrary tools or execute code.

```
Suggested plan fields:
intent: general | college_specific | multi_task | unclear | out_of_scope
requires_rag: boolean
tasks: [{task_id, question, category?, semester?, academic_year?, dependencies: [], suggested_agent}]
execution_mode: direct | parallel | sequential | mixed
clarification_question?: string
```

• Keep simple questions as one task. Initially cap a query at five subtasks.

• Parallelize only tasks with no dependency on one another, with a concurrency limit and per-task timeout.

• Execute dependent tasks only after prerequisite results exist.

• If planner output is invalid or Ollama planning fails, use a safe fallback: direct response for clear greetings; otherwise one conservative RAG task or a clarification request.

• Do not expose chain-of-thought. The UI receives simple progress/status, not private planning reasoning.

## 4. Phase-by-phase backend implementation

### Phase 1 — Environment and foundation

• Create Python virtual environment, requirements file, settings, logging, and local storage paths.

• Configure Ollama model, embedding model, upload limits, allowed file types, timeouts, and retry count.

• Add an Ollama smoke test and basic health check. Keep data/, local models, and .env out of Git.

**Pass condition: packages install, Ollama responds, configuration loads, and a basic test passes.**

## Phase 2 — Basic text RAG and ingestion

• Support PDF/TXT; validate type, size, integrity, and decoding.

• Use PyMuPDF for page-aware text extraction. Use roughly 400–600 token chunks with 50–100 overlap, splitting at headings/paragraphs where possible.

• Store document ID, filename, page, category, effective date/version, content hash, and ingestion timestamp.

• Create local embeddings and persist chunk text/vectors/metadata in ChromaDB. Preserve originals and extracted text locally; track document status in SQLite.

• Retrieve top-k passages and generate an answer using Ollama with filename/page citations.

**Pass condition: a test PDF is indexed and a question gets an evidence-grounded answer with source references.**

## Phase 3 — Local retrieval fallback and version tracking

• Create a local extracted-text cache independent of ChromaDB and BM25 keyword search over it.

• On transient embedding/ChromaDB failure, retry once with timeout, then use BM25.

• If Ollama generation fails, return source passages with a generation-unavailable status, not an invented summary.

• Detect duplicates/updates with content hashes and distinguish current from superseded versions.

**Pass condition: simulated ChromaDB failure uses BM25 and returns source-linked evidence.**

## Phase 4 — Image and scanned-document OCR

• Accept PNG/JPG and OCR scanned PDF pages with no meaningful selectable text.

• Validate images with Pillow and enforce dimensions/size limits. Run RapidOCR and assess usable text.

• If OCR fails or extracts too little, preprocess (resize/grayscale) and retry once.

• Use an Ollama vision model only if the selected model is confirmed to support image input. Keep the text agent's original question alongside OCR output.

**Pass condition: screenshot/scanned-page text can drive retrieval; unreadable images fail safely.**

## Phase 5 — Specialist agents and shared result schema

• Define a Pydantic result with agent, status, answer, sources, failed_stage, error_code, recovery_attempts, and recovery_status.

• Implement text agent with ChromaDB and BM25 fallback. Implement image agent with OCR, preprocessing retry, optional vision fallback, and retrieval from the same knowledge base.

• Require relevant evidence and valid source metadata before treating a college-policy answer as successful.

**Pass condition: both agents return the same validated schema and success/partial/failure tests pass.**

## Phase 6 — Explicit lightweight agentic router

• Implement the four visible routes: general conversation, college-specific query, multi-task query, and known unanswered query.

• Greetings/thanks and suitable general questions bypass all RAG retrieval calls. College-specific facts always use official-document RAG.

• Add scope classification: in-scope, unclear, out-of-scope. Ask for clarification when unclear; decline unrelated requests politely.

• Decompose multi-part queries into bounded subtasks with task IDs, input requirements, dependencies, and suggested agent. Validate every plan with Pydantic.

• Execute independent subtasks concurrently and dependent subtasks sequentially; combine all task results into one final answer.

• Ensure the cache check happens before repeated retrieval for college-specific tasks; do not apply negative cache to general chat.

**Pass condition: unit tests prove “Hi” makes zero retrieval calls; a three-part college question produces one answer; parallel/sequential behavior follows dependencies.**

### Phase 7 — Negative-result cache and invalidation

• Cache only completed retrievals that found insufficient evidence; do not cache ChromaDB/OCR/embedding/Ollama outages as no evidence.

• Use a normalized query plus intent/category/semester/year filters and document-generation ID. Include TTL as a safety net.

• Before retrieval, reuse a cache entry only if unexpired and its generation matches the current indexed knowledge base.

• After an admin successfully indexes/replaces/deactivates a document, increment the knowledge-base generation and invalidate or ignore old negative results.

• Example: Semester 9 fees returns no evidence and is cached. After a new fee document is indexed, the generation changes and the next query performs fresh retrieval.

**Pass condition: cached miss is reused before updates but invalidated after a new document is indexed.**

### Phase 8 — Fault tolerance and cross-agent recovery

• Use bounded retries/timeouts. Text path: ChromaDB/embedding issue → BM25; generation issue → evidence-only fallback with warning.

• Image path: OCR issue → preprocess and retry → optional verified vision fallback.

• Attempt cross-agent recovery only after internal fallback fails and only if the other agent has usable input.

• If image path fails, try text agent with typed question and partial OCR. If text path fails, use image agent only when image/visual input exists.

• If all paths fail or evidence is insufficient, return a safe failure/no-answer result. Never fabricate college facts.

• Return concise failure/recovery status to the user; keep stack traces, secrets, local paths, and raw exception details in protected logs only.

**Pass condition: simulated failures verify each recovery path and bounded stopping behavior.**

### Phase 9 — Evidence, citations, and contradictions

• Give Ollama the query, retrieved passages, source filenames/pages, and task results.

• Require evidence-grounded answers; validate cited sources against actual retrieved metadata.

• Surface conflicting policy versions. Prefer a current version only if approved/effective-date metadata supports it.

• Treat document text as evidence, not system instructions. If evidence is missing, say so; do not guess fees or deadlines.

**Pass condition: citation IDs/pages are valid and unsupported answers are rejected in tests.**

### Phase 10 — API integration and end-to-end tests

• Define stable operations for chat, chat-with-image, document upload/status, and health checks.

• If frontend/backend run separately, expose these through FastAPI; otherwise use direct Python service calls initially.

• Return one response with answer, status, sources, task summary, warnings, agents, recovery details, and request ID.

• Test PDF/TXT, scanned PDF, image, greetings, college questions, multi-task query, negative-cache hit/invalidation, ChromaDB outage, OCR failure, total failure, conflicts, invalid uploads, and restart persistence.

**Pass condition: frontend can integrate and critical happy/failure paths pass.**

## 5. Suggested response object

```
{
"answer": "One combined answer grounded in official sources",
"status": "general | success | partial | no_evidence | degraded | failed",
"sources": [{"filename": "semester_fee.pdf", "page": 3}],
"warnings": [],
"tasks": {"completed": 3, "total": 3},
"recovery": {"attempted": false, "method": null, "status": "not_needed"},
"request_id": "correlation-id"
}
```

Keep internal logs more detailed than user-facing responses. Do not expose stack traces, raw exceptions, local paths, or secrets.

## 6. Privacy and safe operation

• Keep uploads, extracted text, ChromaDB persistence, metadata, local model files, and .env out of version control.

• Avoid logging full student questions or images by default.

• Validate uploaded files and enforce size/dimension limits before processing.

• Protect admin upload routes if the prototype is shared; local-first is not automatically production-grade security.

• Do not answer college-specific factual questions from general model knowledge when official evidence is missing.

## 7. Definition of done

**1.** PDF/TXT uploads are extracted, chunked, indexed, and persisted with source metadata.

**2.** College answers cite actual source filenames/pages; unsupported facts are not invented.

**3.** Image OCR supports screenshots and scanned pages, with safe recovery.

**4.** Greetings bypass retrieval; college questions route to RAG; multi-task queries become one combined answer.

**5.** Independent tasks run in parallel and dependent tasks sequentially.

**6.** Negative results are cached only for the current document generation and invalidated after successful document updates.

**7.** ChromaDB failures trigger BM25 when the local text cache is available; transient failures are not recorded as no evidence.

**8.** Cross-agent recovery is bounded and status is reported honestly.

**9.** Automated tests cover normal flows, failure paths, citations, conflicting versions, and restart persistence.

## 8. Prompt to give Claude

Use this plan and the original project brief as requirements. Inspect my existing repository first. Implement only Phase 1. Give complete files, exact Windows terminal commands, and a test command. Do not create placeholder modules for later phases or modify unrelated working code. Wait for me to verify Phase 1 before moving on.

Basis: original uploaded implementation brief, pages 3–13. Explicit greeting bypass and the negative-result cache are additions from the user's subsequent requirements and screenshot.